

Awesome 🚀 Let's move on to **Module 17: VIEWS**.



SQL CLASS NOTE – MODULE 17: VIEWS

1. Introduction

A **View** in SQL is a **virtual table** based on the result of a query.

- It does **not store data physically**, but stores the **query definition**.
- Whenever you query the view, it dynamically retrieves data from the base tables.

👉 Think of a View as a **saved query** that behaves like a table.

2. Syntax

Create a View

```
CREATE VIEW view_name AS  
SELECT column1, column2, ...  
FROM table_name  
WHERE condition;
```

Query a View

```
SELECT * FROM view_name;
```

Drop a View

```
DROP VIEW view_name;
```

3. Practical Examples with SchoolDB

Example 1: Create a view of all students with their enrollments

```
CREATE VIEW StudentCourses AS
SELECT S.StudentID, S.FirstName, S.LastName, C.CourseName, C.Teacher
FROM Students S
LEFT JOIN Enrollments E ON S.StudentID = E.StudentID
LEFT JOIN Courses C ON E.CourseID = C.CourseID;
```

Usage:

```
SELECT * FROM StudentCourses;
```

Example 2: Create a view of average student age per grade

```
CREATE VIEW GradeAvgAge AS
SELECT GradeLevel, AVG(Age) AS AvgAge
FROM Students
GROUP BY GradeLevel;
```

Usage:

```
SELECT * FROM GradeAvgAge;
```

Example 3: Create a view for courses with total enrollments

```
CREATE VIEW CourseEnrollments AS
SELECT C.CourseID, C.CourseName, COUNT(E.StudentID) AS TotalEnrollments
FROM Courses C
LEFT JOIN Enrollments E ON C.CourseID = E.CourseID
GROUP BY C.CourseID, C.CourseName;
```

Usage:

```
SELECT * FROM CourseEnrollments;
```

Example 4: Create a view of students without enrollments

```
CREATE VIEW StudentsWithoutCourses AS
SELECT S.StudentID, S.FirstName, S.LastName
FROM Students S
WHERE S.StudentID NOT IN (SELECT StudentID FROM Enrollments);
```

Example 5: Update through a view (when possible)

👉 Some views allow `INSERT` or `UPDATE` if based on a **single table without aggregates**.

```
CREATE VIEW StudentBasic AS
SELECT StudentID, FirstName, LastName, Age
FROM Students;
```

```
UPDATE StudentBasic
SET Age = 18
WHERE StudentID = 101;
```

4. Why Use Views?

- **Simplify queries:** Save complex joins/aggregations.
- **Security:** Limit columns exposed to users.
- **Consistency:** Provide a standard format of data.
- **Reusability:** Query the same logic multiple times.

5. Real-Life Use Cases

- **Schools:** Create a view for student performance summary.
- **Banks:** Views for customer balance summaries.
- **E-commerce:** Sales reports per product category.
- **Hospitals:** Patient visit history view.

6. Classwork (Practice)

👉 Create the following views:

1. A view called `StudentCourses` that shows students with their course names and teachers.
2. A view called `GradeAvgAge` showing average age per grade.
3. A view called `CourseEnrollments` showing courses and total number of enrollments.
4. A view called `StudentsWithoutCourses` listing students not enrolled in any course.
5. A view called `TeachersCourses` showing each teacher with the courses they teach.
6. Query each of the views you created using `SELECT * FROM view_name; .`

7. Challenge Question

👉 Create a view called `ActiveGrades` that shows:

- Each `GradeLevel`
- Number of students
- Average age

But **only include grades that have more than 10 students.**

✅ That's the **expanded Module 17: Views.**

👉 Perfect 👍 Let's solve the **Module 17 Classwork + Challenge** step by step with SQL queries.



Solutions to Module 17: VIEWS

1. Create a view called `StudentCourses` that shows students with their course names and teachers.

```
CREATE VIEW StudentCourses AS
SELECT S.StudentID, S.FirstName, S.LastName, C.CourseName, C.Teacher
FROM Students S
LEFT JOIN Enrollments E ON S.StudentID = E.StudentID
LEFT JOIN Courses C ON E.CourseID = C.CourseID;
```

Usage:

```
SELECT * FROM StudentCourses;
```

2. Create a view called `GradeAvgAge` showing average age per grade.

```
CREATE VIEW GradeAvgAge AS
SELECT GradeLevel, AVG(Age) AS AvgAge
FROM Students
GROUP BY GradeLevel;
```

Usage:

```
SELECT * FROM GradeAvgAge;
```

3. Create a view called `CourseEnrollments` showing courses and total number of enrollments.

```
CREATE VIEW CourseEnrollments AS
SELECT C.CourseID, C.CourseName, COUNT(E.StudentID) AS TotalEnrollments
FROM Courses C
LEFT JOIN Enrollments E ON C.CourseID = E.CourseID
GROUP BY C.CourseID, C.CourseName;
```

Usage:

```
SELECT * FROM CourseEnrollments;
```

4. Create a view called `StudentsWithoutCourses` listing students not enrolled in any course.

```
CREATE VIEW StudentsWithoutCourses AS  
SELECT S.StudentID, S.FirstName, S.LastName  
FROM Students S  
WHERE S.StudentID NOT IN (SELECT StudentID FROM Enrollments);
```

Usage:

```
SELECT * FROM StudentsWithoutCourses;
```

5. Create a view called `TeachersCourses` showing each teacher with the courses they teach.

```
CREATE VIEW TeachersCourses AS  
SELECT Teacher, CourseName  
FROM Courses;
```

Usage:

```
SELECT * FROM TeachersCourses;
```

6. Query each of the views created.

```
SELECT * FROM StudentCourses;  
SELECT * FROM GradeAvgAge;  
SELECT * FROM CourseEnrollments;  
SELECT * FROM StudentsWithoutCourses;  
SELECT * FROM TeachersCourses;
```



Challenge Question

👉 Create a view called `ActiveGrades` that shows:

- Each `GradeLevel`
- Number of students
- Average age

But only include grades that have **more than 10 students**.

```
CREATE VIEW ActiveGrades AS
SELECT GradeLevel, COUNT(*) AS StudentCount, AVG(Age) AS AverageAge
FROM Students
GROUP BY GradeLevel
HAVING COUNT(*) > 10;
```

Usage:

```
SELECT * FROM ActiveGrades;
```

📌 This combines **GROUP BY + HAVING** inside a view to pre-filter the results.